Phase 2

DLBDSPBDM01

Project: Build a Data Mart in SQL

Piotr Trybuś

92132989

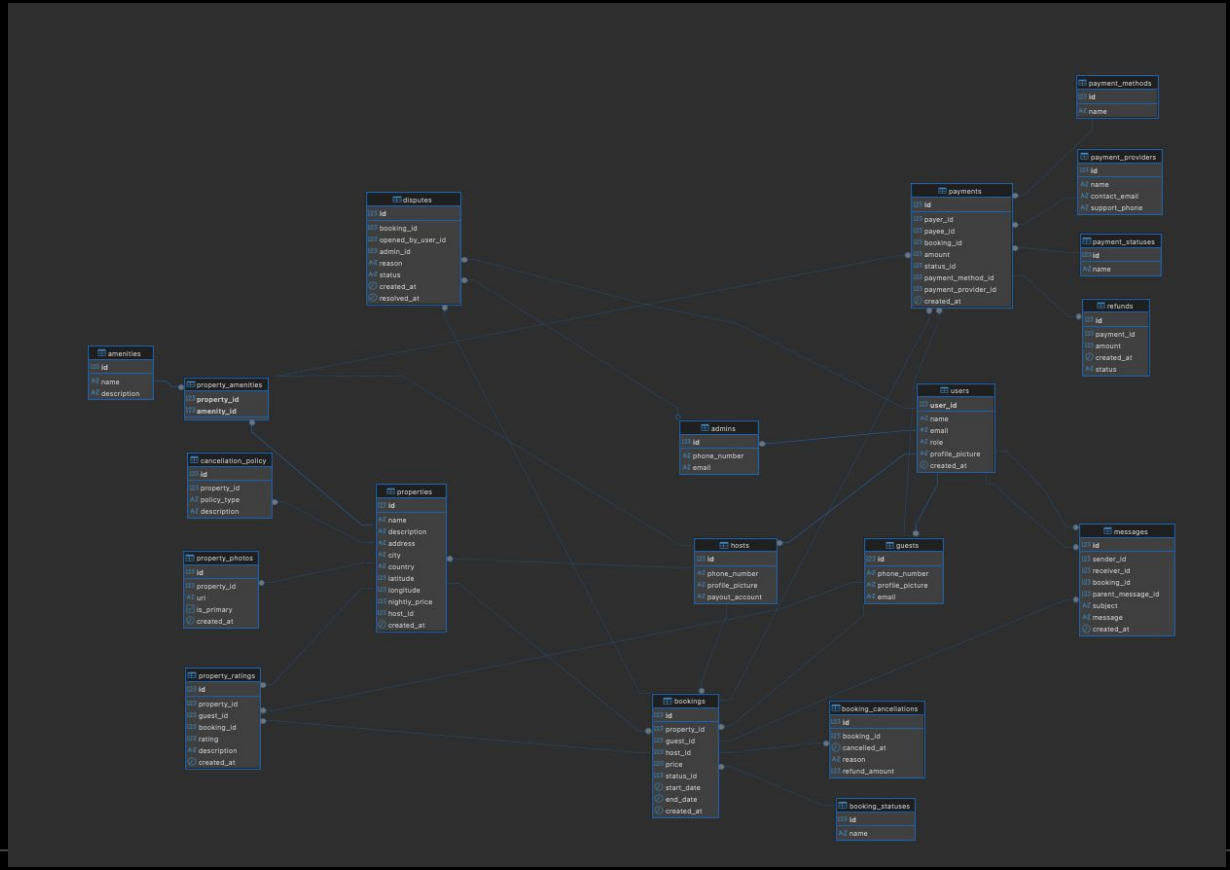
Phase 2 Objectives

- Implement the ER model from Phase 1 in a relational DBMS
- Create all entities and relationships using SQL
- Enforce primary and foreign key constraints
- Populate each table with at least 20 records
- Validate the model using SQL test cases

Technical Environment

- Database Management System: DuckDB
- SQL Client: DBeaver
- Database Schema: booking

Implemented ER Diagram



Entity: users

- **Purpose:**
  Stores all system users (guests, hosts, admins)

```
CREATE TABLE booking.users (
    user_id INT NOT NULL,
    name VARCHAR,
    email VARCHAR,
    role VARCHAR,
    profile_picture VARCHAR,
    created_at TIMESTAMP,
    PRIMARY KEY (user_id)
);
```

- **Test Case**

```
SELECT user_id, name, email, role
FROM booking.users;
```

Results 1

Enter a SQL expression to filter results

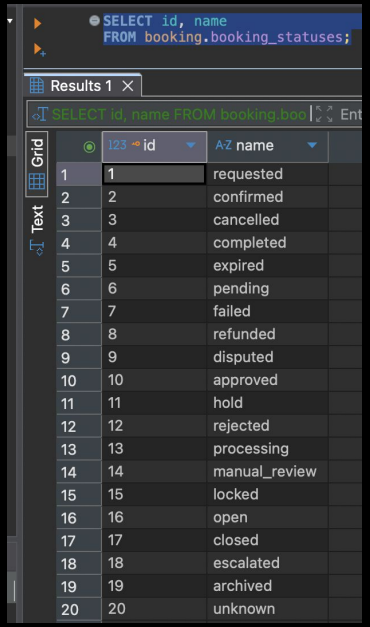
	user_id	name	email	role
1	1	User 01	u01@mail.com	guest
2	2	User 02	u02@mail.com	guest
3	3	User 03	u03@mail.com	guest
4	4	User 04	u04@mail.com	guest
5	5	User 05	u05@mail.com	guest
6	6	User 06	u06@mail.com	host
7	7	User 07	u07@mail.com	host
8	8	User 08	u08@mail.com	host
9	9	User 09	u09@mail.com	host
10	10	User 10	u10@mail.com	host
11	11	User 11	u11@mail.com	admin
12	12	User 12	u12@mail.com	admin
13	13	User 13	u13@mail.com	guest
14	14	User 14	u14@mail.com	guest
15	15	User 15	u15@mail.com	guest
16	16	User 16	u16@mail.com	guest
17	17	User 17	u17@mail.com	guest
18	18	User 18	u18@mail.com	guest
19	19	User 19	u19@mail.com	guest
20	20	User 20	u20@mail.com	guest

Entity: booking_statuses

- **Purpose:**
  Lookup table defining possible booking states

```
CREATE TABLE booking.booking_statuses (
    id INT NOT NULL,
    name VARCHAR NOT NULL,
    PRIMARY KEY (id)
);
```

- **Test Case**

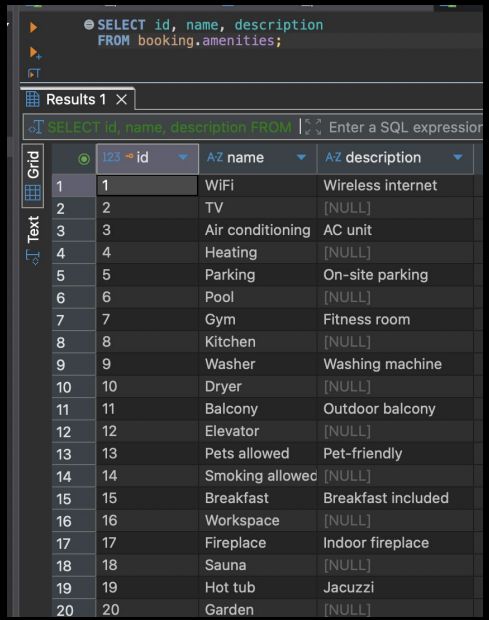


Entity: amenities

- **Purpose:**
  Lookup table describing available amenities

```
CREATE TABLE booking.amenities (
    id INT NOT NULL,
    name VARCHAR NOT NULL,
    description VARCHAR,
    PRIMARY KEY (id)
);
```

- **Test Case**



```
SELECT id, name, description
FROM booking.amenities;
```

Results 1

SELECT id, name, description FROM | Enter a SQL expression

Grid | Text

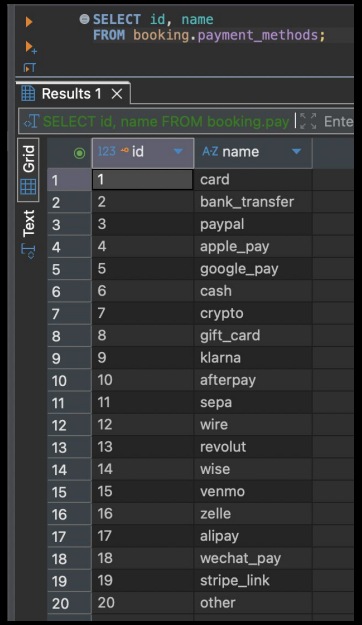
	id	name	description
1	1	WiFi	Wireless internet
2	2	TV	[NULL]
3	3	Air conditioning	AC unit
4	4	Heating	[NULL]
5	5	Parking	On-site parking
6	6	Pool	[NULL]
7	7	Gym	Fitness room
8	8	Kitchen	[NULL]
9	9	Washer	Washing machine
10	10	Dryer	[NULL]
11	11	Balcony	Outdoor balcony
12	12	Elevator	[NULL]
13	13	Pets allowed	Pet-friendly
14	14	Smoking allowed	[NULL]
15	15	Breakfast	Breakfast included
16	16	Workspace	[NULL]
17	17	Fireplace	Indoor fireplace
18	18	Sauna	[NULL]
19	19	Hot tub	Jacuzzi
20	20	Garden	[NULL]

Entity: payment_methods

- **Purpose:**
  Lookup table describing available payment methods

```
CREATE TABLE booking.payment_methods (
    id INT NOT NULL,
    name VARCHAR NOT NULL,
    PRIMARY KEY (id)
);
```

- **Test Case**



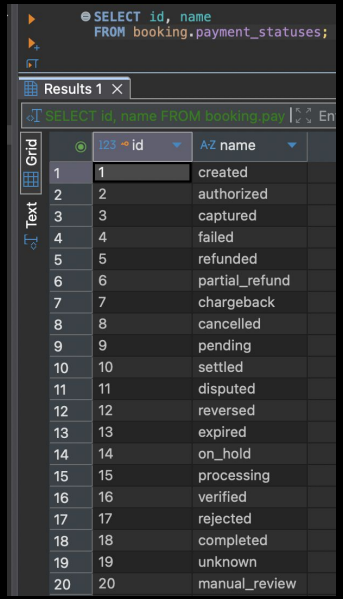
	id	name
1	1	card
2	2	bank_transfer
3	3	paypal
4	4	apple_pay
5	5	google_pay
6	6	cash
7	7	crypto
8	8	gift_card
9	9	klarna
10	10	afterpay
11	11	sepa
12	12	wire
13	13	revolut
14	14	wise
15	15	venmo
16	16	zelle
17	17	alipay
18	18	wechat_pay
19	19	stripe_link
20	20	other

Entity: payment_statuses

- **Purpose:**
  Lookup table describing the statuses in which payments can exist

```
CREATE TABLE booking.payment_statuses (
    id INT NOT NULL,
    name VARCHAR NOT NULL,
    PRIMARY KEY (id)
);
```

- **Test Case**



Entity: guests

- **Purpose:**
  Table describing a user type: guest and the information that is available or required on their account. Required: phone number and email

```sql
CREATE TABLE booking.guests (
    id INT NOT NULL,
    phone_number VARCHAR NOT NULL,
    profile_picture VARCHAR,
    email VARCHAR NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (id) REFERENCES booking.users(user_id)
);
```

- **Test Case**

```sql
SELECT id, phone_number, profile_picture, email
FROM booking.guests;
```

Results 1

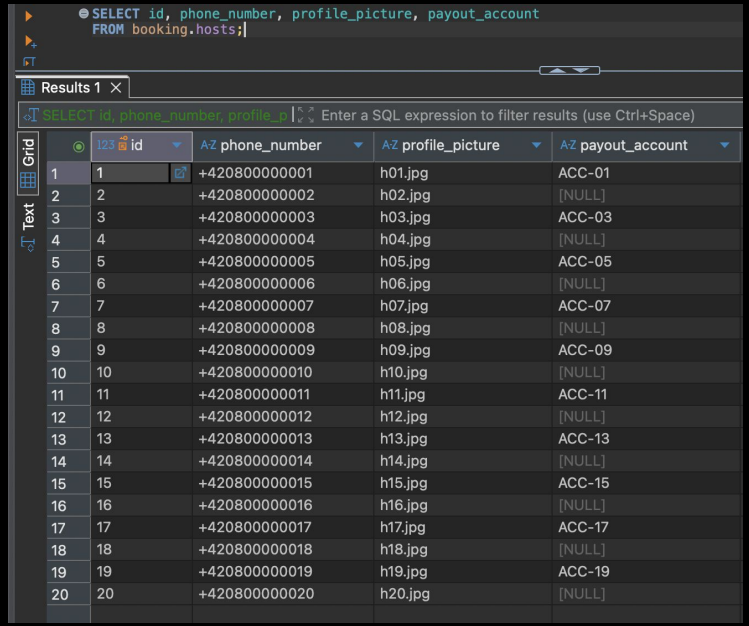
	id	phone_number	profile_picture	email
1	1	+420700000001	g01.jpg	u01@mail.com
2	2	+420700000002	g02.jpg	u02@mail.com
3	3	+420700000003	g03.jpg	u03@mail.com
4	4	+420700000004	g04.jpg	u04@mail.com
5	5	+420700000005	g05.jpg	u05@mail.com
6	6	+420700000006	g06.jpg	u06@mail.com
7	7	+420700000007	g07.jpg	u07@mail.com
8	8	+420700000008	g08.jpg	u08@mail.com
9	9	+420700000009	g09.jpg	u09@mail.com
10	10	+420700000010	g10.jpg	u10@mail.com
11	11	+420700000011	g11.jpg	u11@mail.com
12	12	+420700000012	g12.jpg	u12@mail.com
13	13	+420700000013	g13.jpg	u13@mail.com
14	14	+420700000014	g14.jpg	u14@mail.com
15	15	+420700000015	g15.jpg	u15@mail.com
16	16	+420700000016	g16.jpg	u16@mail.com
17	17	+420700000017	g17.jpg	u17@mail.com
18	18	+420700000018	g18.jpg	u18@mail.com
19	19	+420700000019	g19.jpg	u19@mail.com
20	20	+420700000020	g20.jpg	u20@mail.com

Entity: hosts

- **Purpose:**
  Table describing a user type: host and the information that is available or required on their account. Required: phone number

```
CREATE TABLE booking.hosts (
    id INT NOT NULL,
    phone_number VARCHAR NOT NULL,
    profile_picture VARCHAR,
    payout_account VARCHAR,
    PRIMARY KEY (id),
    FOREIGN KEY (id) REFERENCES booking.users(user_id)
);
```

- **Test Case**



```
SELECT id, phone_number, profile_picture, payout_account
FROM booking.hosts;
```

Results 1

SELECT id, phone_number, profile_p | Enter a SQL expression to filter results (use Ctrl+Space)

Grid | Text

	id	phone_number	profile_picture	payout_account
1	1	+420800000001	h01.jpg	ACC-01
2	2	+420800000002	h02.jpg	[NULL]
3	3	+420800000003	h03.jpg	ACC-03
4	4	+420800000004	h04.jpg	[NULL]
5	5	+420800000005	h05.jpg	ACC-05
6	6	+420800000006	h06.jpg	[NULL]
7	7	+420800000007	h07.jpg	ACC-07
8	8	+420800000008	h08.jpg	[NULL]
9	9	+420800000009	h09.jpg	ACC-09
10	10	+420800000010	h10.jpg	[NULL]
11	11	+420800000011	h11.jpg	ACC-11
12	12	+420800000012	h12.jpg	[NULL]
13	13	+420800000013	h13.jpg	ACC-13
14	14	+420800000014	h14.jpg	[NULL]
15	15	+420800000015	h15.jpg	ACC-15
16	16	+420800000016	h16.jpg	[NULL]
17	17	+420800000017	h17.jpg	ACC-17
18	18	+420800000018	h18.jpg	[NULL]
19	19	+420800000019	h19.jpg	ACC-19
20	20	+420800000020	h20.jpg	[NULL]

Entity: admins

- **Purpose:**
  Table describing a user type: admin and the information that is available or required on their account. Required: phone number

```
CREATE TABLE booking.admins (
  id INT NOT NULL,
  phone_number VARCHAR,
  email VARCHAR,
  PRIMARY KEY (id),
  FOREIGN KEY (id) REFERENCES booking.users(user_id)
);
```

- **Test Case**

```
SELECT id, phone_number, email
FROM booking.admins;
```

Results 1

SELECT id, phone_number, email FRO | Enter a SQL expression to filter

Grid | Text

	id	phone_number	email
1	1	[NULL]	admin01@mail.com
2	2	+420900000002	admin02@mail.com
3	3	[NULL]	[NULL]
4	4	+420900000004	[NULL]
5	5	[NULL]	admin05@mail.com
6	6	+420900000006	[NULL]
7	7	[NULL]	admin07@mail.com
8	8	+420900000008	admin08@mail.com
9	9	[NULL]	[NULL]
10	10	+420900000010	admin10@mail.com
11	11	[NULL]	admin11@mail.com
12	12	+420900000012	[NULL]
13	13	[NULL]	admin13@mail.com
14	14	+420900000014	admin14@mail.com
15	15	[NULL]	[NULL]
16	16	+420900000016	admin16@mail.com
17	17	[NULL]	admin17@mail.com
18	18	+420900000018	[NULL]
19	19	[NULL]	admin19@mail.com
20	20	+420900000020	admin20@mail.com

Entity: properties

- **Purpose:**
  Table storing key data about rental properties, their location, description and related host.

```
CREATE TABLE booking.properties (
    id INT NOT NULL,
    name VARCHAR,
    description VARCHAR,
    address VARCHAR,
    city VARCHAR,
    country VARCHAR,
    latitude NUMERIC,
    longitude NUMERIC,
    nightly_price NUMERIC,
    host_id INT NOT NULL,
    created_at TIMESTAMP,
    PRIMARY KEY (id),
    FOREIGN KEY (host_id) REFERENCES booking.hosts(id)
 );
```

- **Test Case**

```
SELECT
    id,
    name,
    description,
    address,
    city,
    country,
    latitude,
    longitude,
    nightly_price,
    host_id,
    created_at

FROM booking.properties;
```

Results 1

Enter a SQL expression to filter results (use Ctrl+Space)

	id	name	description	address	city	country	latitude	longitude	nightly_price	host_id
1	1	Prop 01	Modern studio	Addr 01	Prague	CZ	50.087	14.421	110	1
2	2	Prop 02	[NULL]	Addr 02	Prague	CZ	[NULL]	[NULL]	95	2
3	3	Prop 03	Quiet flat	Addr 03	Brno	CZ	49.195	16.607	[NULL]	3
4	4	Prop 04	[NULL]	Addr 04	Brno	CZ	49.2	[NULL]	90	4
5	5	Prop 05	City view	Addr 05	Ostrava	CZ	[NULL]	18.262	120	5
6	6	Prop 06	Budget room	Addr 06	Plzen	CZ	[NULL]	[NULL]	75	6
7	7	Prop 07	Family apt	Addr 07	Plzen	CZ	49.747	13.377	[NULL]	7
8	8	Prop 08	Central	Addr 08	Prague	CZ	50.09	14.4	150	8
9	9	Prop 09	[NULL]	Addr 09	Prague	CZ	[NULL]	[NULL]	80	9
10	10	Prop 10	Luxury	Addr 10	Prague	CZ	50.1	14.45	220	10
11	11	Prop 11	[NULL]	Addr 11	Brno	CZ	[NULL]	[NULL]	85	11
12	12	Prop 12	Renovated	Addr 12	Brno	CZ	49.21	16.62	130	12
13	13	Prop 13	Near park	Addr 13	Ostrava	CZ	[NULL]	18.3	[NULL]	13
14	14	Prop 14	[NULL]	Addr 14	Ostrava	CZ	49.82	[NULL]	90	14
15	15	Prop 15	Business	Addr 15	Prague	CZ	50.08	14.41	160	15
16	16	Prop 16	[NULL]	Addr 16	Plzen	CZ	[NULL]	[NULL]	100	16
17	17	Prop 17	Bright	Addr 17	Plzen	CZ	49.76	13.39	[NULL]	17
18	18	Prop 18	[NULL]	Addr 18	Prague	CZ	[NULL]	[NULL]	115	18
19	19	Prop 19	Cozy	Addr 19	Brno	CZ	49.19	16.6	98	19
20	20	Prop 20	[NULL]	Addr 20	Ostrava	CZ	[NULL]	[NULL]	105	20

Entity: cancellation_policy

- **Purpose:**
  Table storing key data about each property's cancellation policy..

```
CREATE TABLE booking.cancellation_policy (
    id INT NOT NULL,
    property_id INT NOT NULL,
    policy_type VARCHAR,
    description VARCHAR,
    PRIMARY KEY (id),
    FOREIGN KEY (property_id) REFERENCES booking.properties(id)
);
```

- **Test Case**

```
SELECT
    id,
    property_id,
    policy_type,
    description
FROM booking.cancellation_policy;
```

Results 1

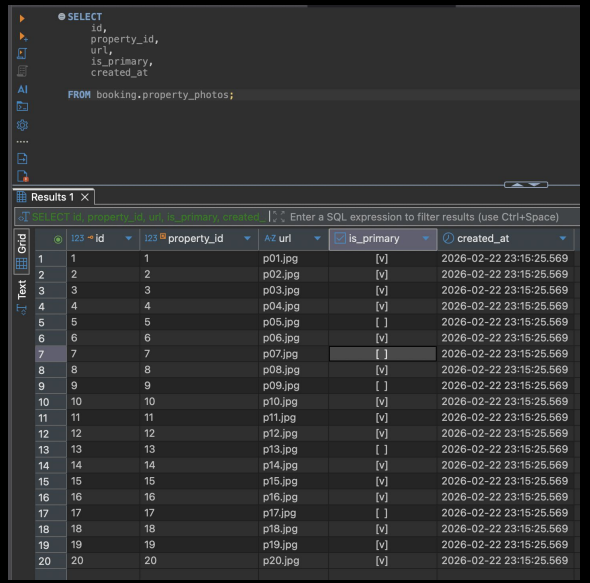
	id	property_id	policy_type	description
1	1	1	flexible	Cancel up to 24h prior
2	2	2	moderate	[NULL]
3	3	3	strict	No refunds within 7 days
4	4	4	flexible	[NULL]
5	5	5	moderate	50% refund up to 3 days
6	6	6	strict	[NULL]
7	7	7	flexible	Free cancellation window
8	8	8	moderate	[NULL]
9	9	9	strict	Non-refundable rate
10	10	10	flexible	[NULL]
11	11	11	moderate	Partial refund
12	12	12	strict	[NULL]
13	13	13	flexible	24h window
14	14	14	moderate	[NULL]
15	15	15	strict	No changes allowed
16	16	16	flexible	[NULL]
17	17	17	moderate	Conditions apply
18	18	18	strict	[NULL]
19	19	19	flexible	Free
20	20	20	moderate	[NULL]

Entity: property_photos

- **Purpose:**
  Table storing the reference to the photos of each property, their respective URL and whether they're primary

```
CREATE TABLE booking.property_photos (
    id INT NOT NULL,
    property_id INT NOT NULL,
    url VARCHAR NOT NULL,
    is_primary BOOLEAN,
    created_at TIMESTAMP,
    PRIMARY KEY (id),
    FOREIGN KEY (property_id) REFERENCES booking.properties(id)
);
```

- **Test Case**

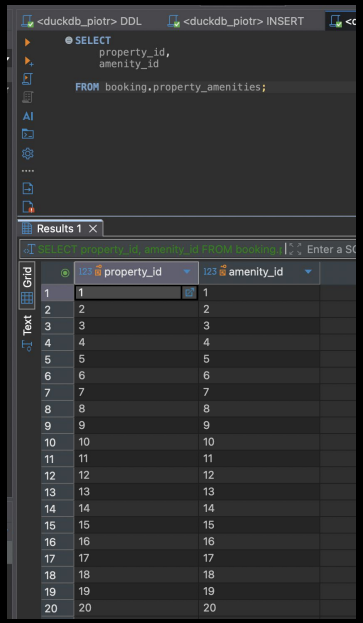


Entity: property_amenities

- **Purpose:**
  Table linking properties to amenities.

```
CREATE TABLE booking.property_amenities (
    property_id INT NOT NULL,
    amenity_id INT NOT NULL,
    PRIMARY KEY (property_id, amenity_id),
    FOREIGN KEY (property_id) REFERENCES booking.properties(id),
    FOREIGN KEY (amenity_id) REFERENCES booking.amenities(id)
);
```

- **Test Case**

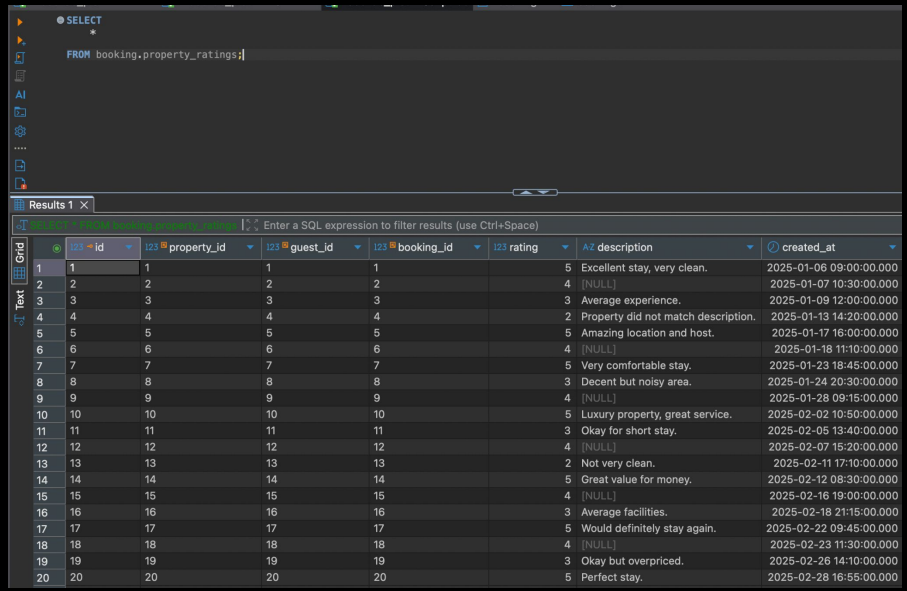


Entity: property_ratings

- **Purpose:**
  Table linking properties and bookings to ratings..

```
CREATE TABLE booking.property_ratings (
    id INT,
    property_id INT NOT NULL,
    guest_id INT NOT NULL,
    booking_id INT NOT NULL,
    rating INT NOT NULL,
    description VARCHAR,
    created_at TIMESTAMP,
    PRIMARY KEY (id),
    FOREIGN KEY (property_id) REFERENCES booking.properties(id),
    FOREIGN KEY (guest_id) REFERENCES booking.guests(id),
    FOREIGN KEY (booking_id) REFERENCES booking.bookings(id)
    );
```

- **Test Case**



```
SELECT
    *
FROM booking.property_ratings;
```

Results 1

Enter a SQL expression to filter results (use Ctrl+Space)

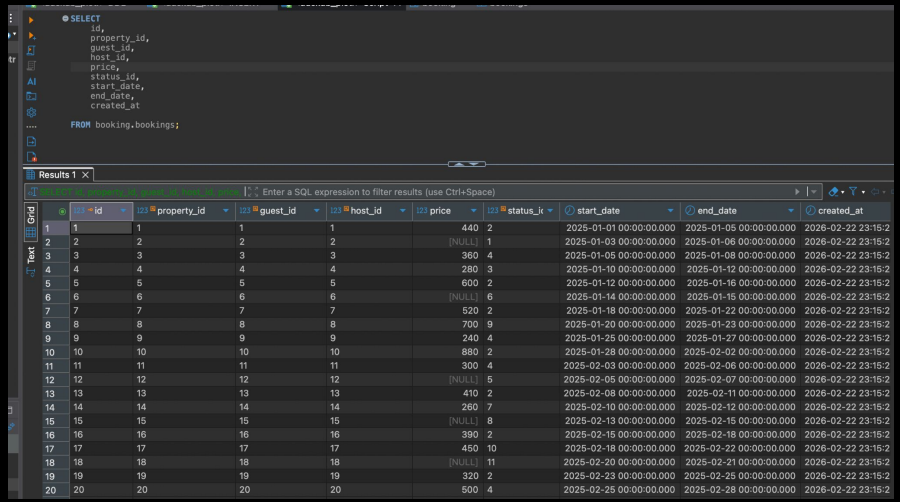
	id	property_id	guest_id	booking_id	rating	description	created_at
1	1	1	1	1	5	Excellent stay, very clean.	2025-01-06 09:00:00.000
2	2	2	2	2	4	[NULL]	2025-01-07 10:30:00.000
3	3	3	3	3	3	Average experience.	2025-01-09 12:00:00.000
4	4	4	4	4	2	Property did not match description.	2025-01-13 14:20:00.000
5	5	5	5	5	5	Amazing location and host.	2025-01-17 16:00:00.000
6	6	6	6	6	4	[NULL]	2025-01-18 11:10:00.000
7	7	7	7	7	5	Very comfortable stay.	2025-01-23 18:45:00.000
8	8	8	8	8	3	Decent but noisy area.	2025-01-24 20:30:00.000
9	9	9	9	9	4	[NULL]	2025-01-28 09:15:00.000
10	10	10	10	10	5	Luxury property, great service.	2025-02-02 10:50:00.000
11	11	11	11	11	3	Okay for short stay.	2025-02-05 13:40:00.000
12	12	12	12	12	4	[NULL]	2025-02-07 15:20:00.000
13	13	13	13	13	2	Not very clean.	2025-02-11 17:10:00.000
14	14	14	14	14	5	Great value for money.	2025-02-12 08:30:00.000
15	15	15	15	15	4	[NULL]	2025-02-16 19:00:00.000
16	16	16	16	16	3	Average facilities.	2025-02-18 21:15:00.000
17	17	17	17	17	5	Would definitely stay again.	2025-02-22 09:45:00.000
18	18	18	18	18	4	[NULL]	2025-02-23 11:30:00.000
19	19	19	19	19	3	Okay but overpriced.	2025-02-26 14:10:00.000
20	20	20	20	20	5	Perfect stay.	2025-02-28 16:55:00.000

Entity: bookings

- **Purpose:**
  Key table working as the main source of booking related data, including the host, guest, linked property, time periods and price

```
CREATE TABLE booking.bookings (
    id INT NOT NULL,
    property_id INT NOT NULL,
    guest_id INT NOT NULL,
    host_id INT NOT NULL,
    price NUMERIC,
    status_id INT NOT NULL,
    start_date DATE,
    end_date DATE,
    created_at TIMESTAMP,
    PRIMARY KEY (id),
    FOREIGN KEY (guest_id) REFERENCES booking.guests(id),
    FOREIGN KEY (host_id) REFERENCES booking.hosts(id),
    FOREIGN KEY (property_id) REFERENCES booking.properties(id),
    FOREIGN KEY (status_id) REFERENCES booking.booking_statuses(id)
);
```

- **Test Case**

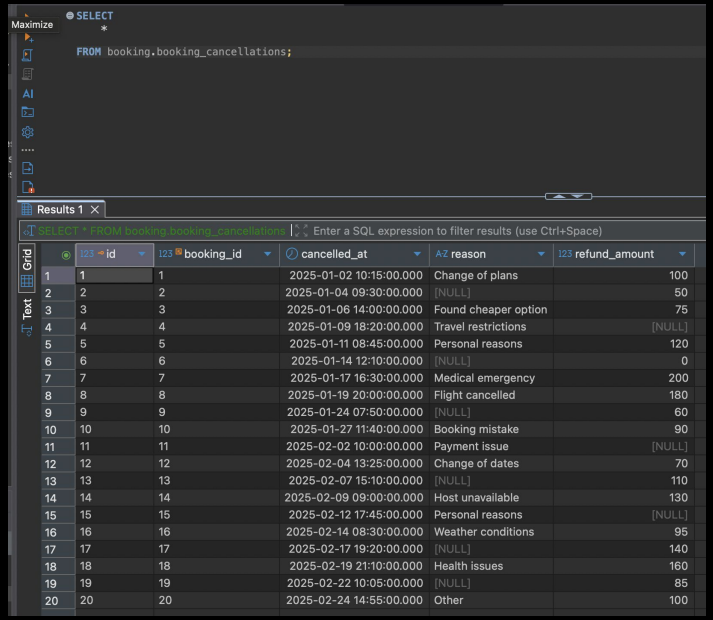


Entity: booking_cancellations

- **Purpose:**
  Specialized table with data on reasons related to booking cancellations

```
CREATE TABLE booking.booking_cancellations (
    id INT,
    booking_id INT NOT NULL,
    cancelled_at TIMESTAMP NOT NULL,
    reason VARCHAR,
    refund_amount NUMERIC,
    PRIMARY KEY (id),
    FOREIGN KEY(booking_id) REFERENCES booking.bookings(id)
    );
```

- **Test Case**



	id	booking_id	cancelled_at	reason	refund_amount
1	1	1	2025-01-02 10:15:00.000	Change of plans	100
2	2	2	2025-01-04 09:30:00.000	[NULL]	50
3	3	3	2025-01-06 14:00:00.000	Found cheaper option	75
4	4	4	2025-01-09 18:20:00.000	Travel restrictions	[NULL]
5	5	5	2025-01-11 08:45:00.000	Personal reasons	120
6	6	6	2025-01-14 12:10:00.000	[NULL]	0
7	7	7	2025-01-17 16:30:00.000	Medical emergency	200
8	8	8	2025-01-19 20:00:00.000	Flight cancelled	180
9	9	9	2025-01-24 07:50:00.000	[NULL]	60
10	10	10	2025-01-27 11:40:00.000	Booking mistake	90
11	11	11	2025-02-02 10:00:00.000	Payment issue	[NULL]
12	12	12	2025-02-04 13:25:00.000	Change of dates	70
13	13	13	2025-02-07 15:10:00.000	[NULL]	110
14	14	14	2025-02-09 09:00:00.000	Host unavailable	130
15	15	15	2025-02-12 17:45:00.000	Personal reasons	[NULL]
16	16	16	2025-02-14 08:30:00.000	Weather conditions	95
17	17	17	2025-02-17 19:20:00.000	[NULL]	140
18	18	18	2025-02-19 21:10:00.000	Health issues	160
19	19	19	2025-02-22 10:05:00.000	[NULL]	85
20	20	20	2025-02-24 14:55:00.000	Other	100

Entity: disputes

- **Purpose:**
  Disputes related to user requested customer support disputes and their details.

```sql
CREATE TABLE booking.disputes (
    id INT NOT NULL,
    booking_id INT NOT NULL,
    opened_by_user_id INT NOT NULL,
    admin_id INT,
    reason VARCHAR NOT NULL,
    status VARCHAR,
    created_at TIMESTAMP,
    resolved_at TIMESTAMP,
    PRIMARY KEY (id),
    FOREIGN KEY (booking_id) REFERENCES booking.bookings(id),
    FOREIGN KEY (opened_by_user_id) REFERENCES booking.users(user_id),
    FOREIGN KEY (admin_id) REFERENCES booking.admins(id)
);
```

- **Test Case**

```sql
SELECT
    id,
    booking_id,
    opened_by_user_id,
    admin_id,
    reason,
    status,
    created_at,
    resolved_at
FROM booking.disputes;
```

Results 1

Enter a SQL expression to filter results (use Ctrl+Space)

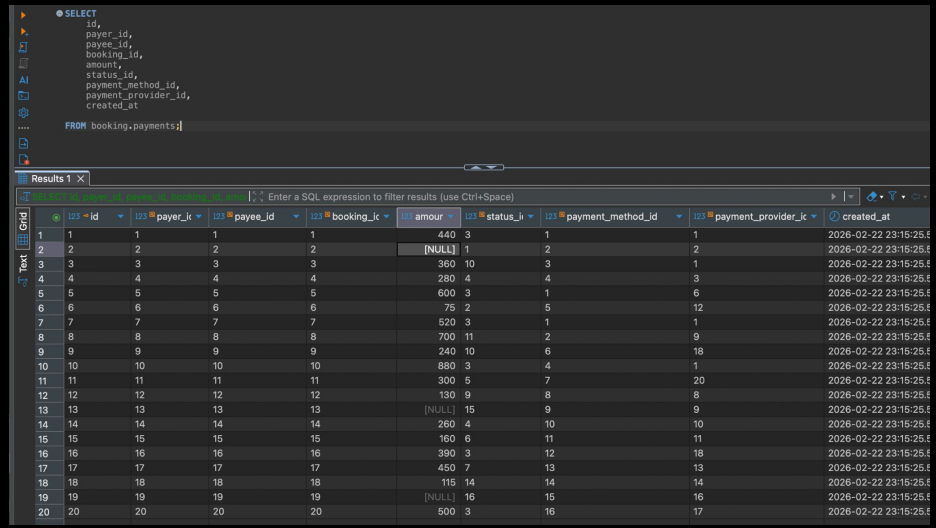
	id	booking_id	opened_by_user_id	admin_id	reason	status	created_at	resolved_at
1	1	1	1	1	Charge disputed	open	2026-02-22 23:15:25.604	[NULL]
2	2	2	2	[NULL]	Host no-show	open	2026-02-22 23:15:25.604	[NULL]
3	3	3	3	2	Property mismatch	escalated	2026-02-22 23:15:25.604	[NULL]
4	4	4	4	3	Refund request	closed	2026-02-22 23:15:25.604	2026-02-22 23:15:25.604
5	5	5	5	[NULL]	Payment failed	open	2026-02-22 23:15:25.604	[NULL]
6	6	6	6	4	Incorrect amount	closed	2026-02-22 23:15:25.604	2026-02-22 23:15:25.604
7	7	7	7	[NULL]	Noise complaint	open	2026-02-22 23:15:25.604	[NULL]
8	8	8	8	5	Security deposit	escalated	2026-02-22 23:15:25.604	[NULL]
9	9	9	9	[NULL]	Cancellation issue	open	2026-02-22 23:15:25.604	[NULL]
10	10	10	10	6	Late check-in	closed	2026-02-22 23:15:25.604	2026-02-22 23:15:25.604
11	11	11	11	[NULL]	Double charge	open	2026-02-22 23:15:25.604	[NULL]
12	12	12	12	7	Damages dispute	open	2026-02-22 23:15:25.604	[NULL]
13	13	13	13	[NULL]	Support request	open	2026-02-22 23:15:25.604	[NULL]
14	14	14	14	8	Chargeback	escalated	2026-02-22 23:15:25.604	[NULL]
15	15	15	15	[NULL]	Refund timing	open	2026-02-22 23:15:25.604	[NULL]
16	16	16	16	9	Host payout	closed	2026-02-22 23:15:25.604	2026-02-22 23:15:25.604
17	17	17	17	[NULL]	Listing inaccurate	open	2026-02-22 23:15:25.604	[NULL]
18	18	18	18	10	Payment reversal	open	2026-02-22 23:15:25.604	[NULL]
19	19	19	19	[NULL]	Booking cancelled	closed	2026-02-22 23:15:25.604	2026-02-22 23:15:25.604
20	20	20	20	11	Other	open	2026-02-22 23:15:25.604	[NULL]

Entity: payments

- **Purpose:**
  Key table containing all payment data and related details.

```sql
CREATE TABLE booking.payments (
    id INT NOT NULL,
    payer_id INT NOT NULL,
    payee_id INT NOT NULL,
    booking_id INT NOT NULL,
    amount NUMERIC,
    status_id INT NOT NULL,
    payment_method_id INT NOT NULL,
    payment_provider_id INT NOT NULL,
    created_at TIMESTAMP,
    PRIMARY KEY (id),
    FOREIGN KEY (payer_id) REFERENCES booking.guests(id),
    FOREIGN KEY (payee_id) REFERENCES booking.hosts(id),
    FOREIGN KEY (booking_id) REFERENCES booking.bookings(id),
    FOREIGN KEY (payment_method_id) REFERENCES booking.payment_methods(id),
    FOREIGN KEY (status_id) REFERENCES booking.payment_statuses(id),
    FOREIGN KEY (payment_provider_id) REFERENCES booking.payment_providers(id)
);
```

- **Test Case**



```sql
SELECT
    id,
    payer_id,
    payee_id,
    booking_id,
    amount,
    status_id,
    payment_method_id,
    payment_provider_id,
    created_at
FROM booking.payments;
```

Results 1

Enter a SQL expression to filter results (use Ctrl+Space)

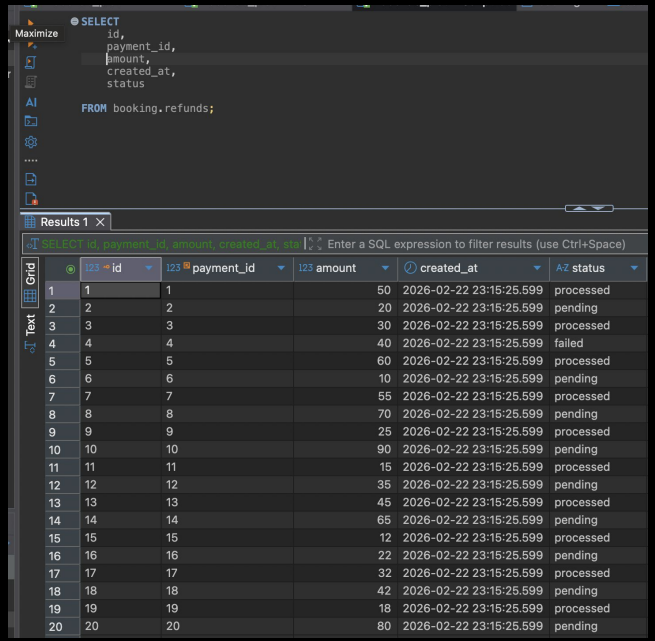
	id	payer_id	payee_id	booking_id	amount	status_id	payment_method_id	payment_provider_id	created_at
1	1	1	1	1	440	3	1	1	2026-02-22 23:15:25.5
2	2	2	2	2	[NULL]	1	2	2	2026-02-22 23:15:25.5
3	3	3	3	3	360	10	3	1	2026-02-22 23:15:25.5
4	4	4	4	4	280	4	4	3	2026-02-22 23:15:25.5
5	5	5	5	5	600	3	1	6	2026-02-22 23:15:25.5
6	6	6	6	6	75	2	5	12	2026-02-22 23:15:25.5
7	7	7	7	7	520	3	1	1	2026-02-22 23:15:25.5
8	8	8	8	8	700	11	2	9	2026-02-22 23:15:25.5
9	9	9	9	9	240	10	6	18	2026-02-22 23:15:25.5
10	10	10	10	10	880	3	4	1	2026-02-22 23:15:25.5
11	11	11	11	11	300	5	7	20	2026-02-22 23:15:25.5
12	12	12	12	12	130	9	8	8	2026-02-22 23:15:25.5
13	13	13	13	13	[NULL]	15	9	9	2026-02-22 23:15:25.5
14	14	14	14	14	260	4	10	10	2026-02-22 23:15:25.5
15	15	15	15	15	160	6	11	11	2026-02-22 23:15:25.5
16	16	16	16	16	390	3	12	18	2026-02-22 23:15:25.5
17	17	17	17	17	450	7	13	13	2026-02-22 23:15:25.5
18	18	18	18	18	115	14	14	14	2026-02-22 23:15:25.5
19	19	19	19	19	[NULL]	16	15	16	2026-02-22 23:15:25.5
20	20	20	20	20	500	3	16	17	2026-02-22 23:15:25.5

Entity: refunds

- **Purpose:**
  Specialized table with data on refunds.

```sql
CREATE TABLE booking.refunds (
    id INT NOT NULL,
    payment_id INT NOT NULL,
    amount NUMERIC NOT NULL,
    created_at TIMESTAMP,
    status VARCHAR,
    PRIMARY KEY (id),
    FOREIGN KEY (payment_id) REFERENCES booking.payments(id)
);
```

- **Test Case**



	id	payment_id	amount	created_at	status
1	1	1	50	2026-02-22 23:15:25.599	processed
2	2	2	20	2026-02-22 23:15:25.599	pending
3	3	3	30	2026-02-22 23:15:25.599	processed
4	4	4	40	2026-02-22 23:15:25.599	failed
5	5	5	60	2026-02-22 23:15:25.599	processed
6	6	6	10	2026-02-22 23:15:25.599	pending
7	7	7	55	2026-02-22 23:15:25.599	processed
8	8	8	70	2026-02-22 23:15:25.599	pending
9	9	9	25	2026-02-22 23:15:25.599	processed
10	10	10	90	2026-02-22 23:15:25.599	pending
11	11	11	15	2026-02-22 23:15:25.599	processed
12	12	12	35	2026-02-22 23:15:25.599	pending
13	13	13	45	2026-02-22 23:15:25.599	processed
14	14	14	65	2026-02-22 23:15:25.599	pending
15	15	15	12	2026-02-22 23:15:25.599	processed
16	16	16	22	2026-02-22 23:15:25.599	pending
17	17	17	32	2026-02-22 23:15:25.599	processed
18	18	18	42	2026-02-22 23:15:25.599	pending
19	19	19	18	2026-02-22 23:15:25.599	processed
20	20	20	80	2026-02-22 23:15:25.599	pending

Cross-table relationships

- **Verification of join conditions** on select tables: